

撷码可达鸭编程
XIE MA CODUCK

撷码金牌学院

CSP防爆“0”手册

NOI金银牌大佬倾情打造

让每个孩子因科技特长而受益

目录

试机注意事项 1
1. 检查硬件设备 1
2. 检查编译器以及环境 1
3. 调整字体和大小 3
4. 检查文件后缀名(.cpp）是否显示 4
考试流程注意事项 7
1. 考试开始即刻：按照要求的格式建立好文件夹和 cpp 文件 7
2. 交卷前：再次检查文件格式以及 freopen、删除多余文件 7
3. 代码检查：时空限制检查、数据范围、变量名 7
调试相关 10
1. 大样例的检查与比较（windows 下的 fc 命令） 10
2. DevC++中的调试工具 10
3. 其他调试方法 11
常见报错案例 13
1. 常见编译错误（介绍一些报错信息的单词） 13
2. 常见爆零例子 14
3. 题目中的 corner 17
进阶知识 18
1. 进阶技巧：对拍 18
2. 常见的判题机的判题逻辑 19
考试策略与心态调整 20
1. 良好的行为习惯 20
2. 时间分配 20
3. 做题策略与骗分 20
4. 写代码的注意事项 20
5. 心态调整 21

试机注意事项

1. 检查硬件设备

(1) 检查硬件设备是否能够正常使用：键盘、鼠标、屏幕。

a. **鼠标**的左右键、滚轮

b. **键盘**的各个按键是否可以正常使用：可以打开 devc++，把每个键都按一遍看是否能打出来

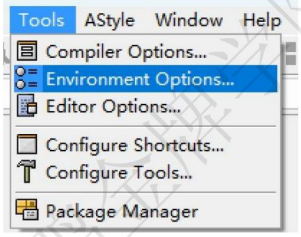
注意**电源插排**是否在自己附近，注意不要不小心弄掉电源导致断电，把**水杯**放到指定位置

(2) 再次检查身上没有电子产品，比如手表、手机、U 盘等，**不要带到考场中。一定要注意！**

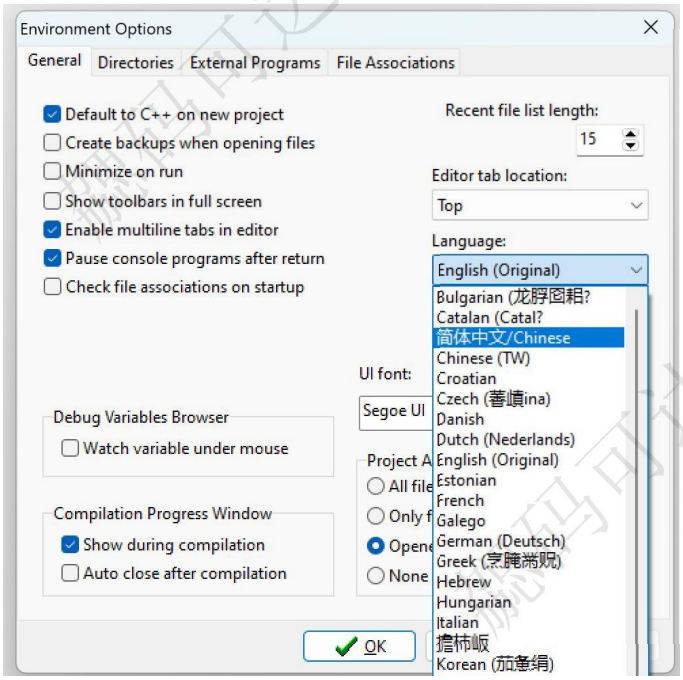
2. 检查编译器以及环境

(1) 将 DevC++语言调整为中文：

a. 打开 Dev，找到 **工具[T]** 或者 **Tools[T]**：



b. 选择 **Language** 为**简体中文**/Chinese：

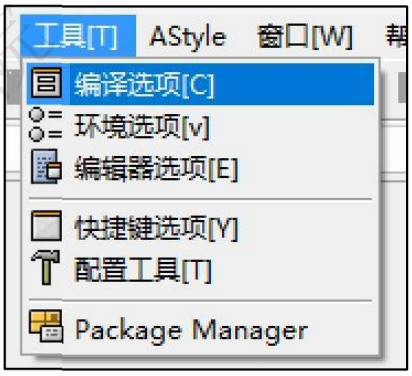


(2) 检查 C++14 的编译选项

a) 打开 Dev，找到 **工具[T]** 或者 **Tools[T]**：



b) 点击菜单中的第一项**编译选项[C]**：



c) 找到考试时发的题面第一页 C++编译选项（以 CSP-J2023 为例）

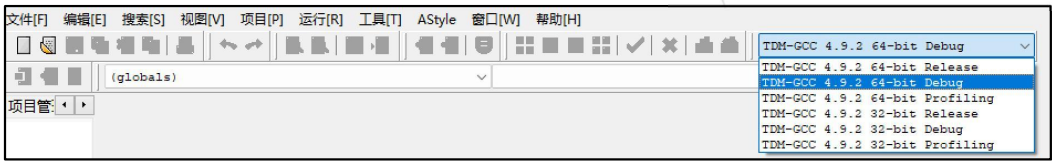
编译选项

对于 C++ 语言	-O2 -std=c++14 -static

d) 将参数填入对应位置，并勾选，确定：



(3) debug 开始前，检查是否调整好 devc++：选择 **64-bit Debug**



(4) 写代码编译运行，查看是否能够正常运行：试一试 **sort、memset** 等函数，以及 **set、map** 等**常用的 STL 容器是否能正常使用。**

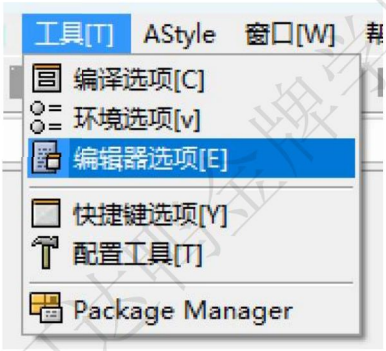
3. 调整字体和大小

(1) 字体：

点击**工具[T]**：



选择**编译器选项[E]**：



点击“显示”，选择 Consolas 字体，点击确定即可：



(2) 字体大小：长按 ctrl 键，鼠标滚轮向上滑动

(3) 检查相似字体：

输入下面这段代码，检查比较容易混淆的字是否区分明显：

```
int iIl1oO0=1;
cout<<iIl1oO0;
```

尤其是注意这种字体中的逗号、点、1lIoO0（数字一、i 的大写形式、L 的小写形式、oO 这个字母的大小写形式、数字 0）这种相似字符。如果区分不明显，需要按照（1）的方法做调整。

4. 检查文件后缀名(.cpp) 是否显示

检查系统是否默认显示后缀名，防止写成类似 apple**.cpp.cpp** 这样的格式。

(1) win10

打开文件资源管理器，查看->显示->文件扩展名

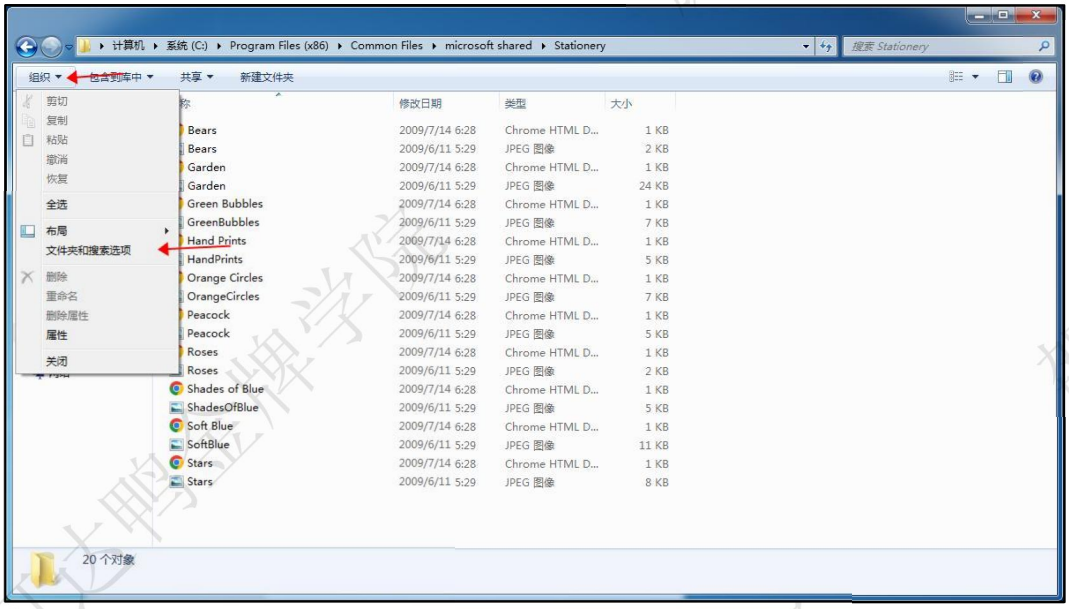


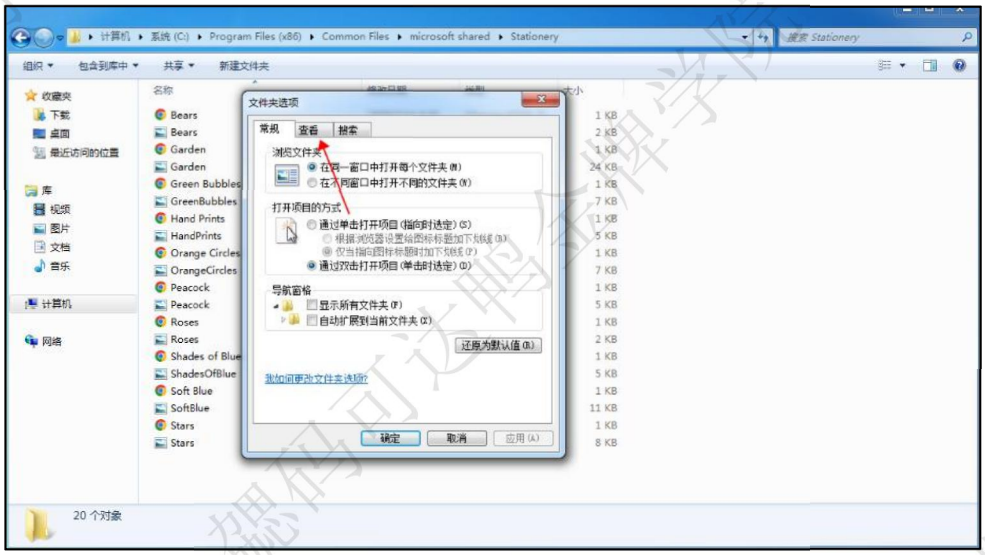
(2) win11

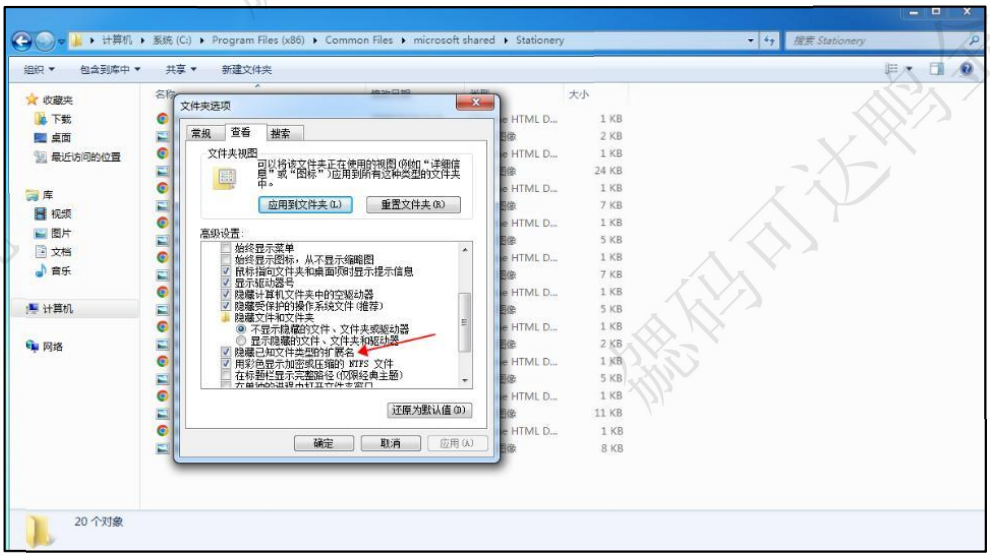
打开文件资源管理器，查看->显示->文件扩展名

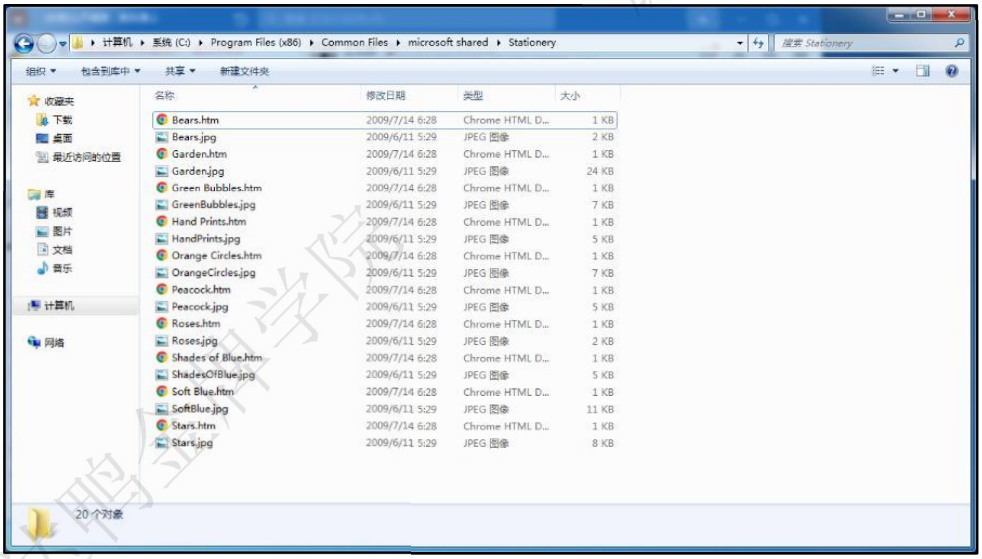


(3) win7：组织->文件夹和搜索选项->查看->隐藏已知文件类型的扩展名(取消勾选)->确定


计算机 ▸ 系统 (C:) ▸ Program Files (x86) ▸ Common Files ▸ microsoft shared ▸ Stationery
组织
包含到库中
共享
新建文件夹
剪切
复制
粘贴
撤消
恢复
全选
布局
文件夹和搜索选项
删除
重命名
删除属性
属性
关闭
名称
修改日期
类型
大小
Bears 2009/7/14 6:28 Chrome HTML D... 1 KB
Bears 2009/6/11 5:29 JPEG 图像 2 KB
Garden 2009/7/14 6:28 Chrome HTML D... 1 KB
Garden 2009/6/11 5:29 JPEG 图像 24 KB
Green Bubbles 2009/7/14 6:28 Chrome HTML D... 1 KB
GreenBubbles 2009/6/11 5:29 JPEG 图像 7 KB
Hand Prints 2009/7/14 6:28 Chrome HTML D... 1 KB
HandPrints 2009/6/11 5:29 JPEG 图像 5 KB
Orange Circles 2009/7/14 6:28 Chrome HTML D... 1 KB
OrangeCircles 2009/6/11 5:29 JPEG 图像 7 KB
Peacock 2009/7/14 6:28 Chrome HTML D... 1 KB
Peacock 2009/6/11 5:29 JPEG 图像 5 KB
Roses 2009/7/14 6:28 Chrome HTML D... 1 KB
Roses 2009/6/11 5:29 JPEG 图像 2 KB
Shades of Blue 2009/7/14 6:28 Chrome HTML D... 1 KB
ShadesOfBlue 2009/6/11 5:29 JPEG 图像 5 KB
Soft Blue 2009/7/14 6:28 Chrome HTML D... 1 KB
SoftBlue 2009/6/11 5:29 JPEG 图像 11 KB
Stars 2009/7/14 6:28 Chrome HTML D... 1 KB
Stars 2009/6/11 5:29 JPEG 图像 8 KB
20 个对象


文件夹选项
常规
查看
搜索
浏览文件夹
在同一窗口中打开每个文件夹(M)
在不同窗口中打开不同的文件夹(N)
打开项目的方式
通过单击打开项目(指向时选定)(C)
通过双击打开项目(单击时选定)(D)
导航窗格
显示所有文件夹(F)
自动扩展到当前文件夹(X)
还原为默认值(R)
确定
取消
应用(A)
20 个对象


文件夹选项
常规
查看
搜索
文件夹视图
应用到文件夹(L)
重置文件夹(R)
高级设置:
隐藏已知文件类型的扩展名
还原为默认值(D)
确定
取消
应用(A)
20 个对象



如果不确定，也可以新建一个文件（比如 txt 文本文档），看一下默认情况下是否显示后缀名：



考试流程注意事项

1. 考试开始即刻：按照要求的格式建立好文件夹和 cpp 文件

打开题目，查看好第一页的四道题英文名，按要求建好文件夹。记住文件夹保存的位置，最好放在桌面防止找不到！**不要等写完了代码再保存！**否则最后整合的时候，很容易出错。

正式比赛时，文件格式参考如下，**具体的文件格式以各省的通知为准（比如四川省采用程序回收系统，不需要建文件夹）**



2. 交卷前：再次检查文件格式以及 freopen、删除多余文件

牢记 freopen 的写法，main 函数内，最开始两行要写：

```
freopen("题目英文名.in","r",stdin);
freopen("题目英文名.out","w",stdout);
```

把**题目英文名**替换成对应题目的英文名。后面的 r 代表 read-only，只读。w 代表 writable，可写。

有些同学担心忘记会提前写上并注释，注意提交前再次检查：freopen 有没有被注释，**一定要记得取消注释 freopen!**

还有一些同学，因为在测试时为了方便从样例文件读入，把代码写成了题目名+序号的格式：

```
freopen("题目英文名 1.in","r",stdin); ✕✕✕
```

这种情况下忘记删除序号，也会导致文件读写失败！

未按规定建立子目录、建立的子目录名出现错误或提交的源程序没有存放在相应的子目录下等都会导致选手成绩为 0 分，责任由选手承担。全国要求：选手试题目录下，不得包含除有效答案以外的任何文件和目录。

交卷之前记得检查多余内容并删除。

3. 代码检查：时空限制检查、数据范围、变量名

认真阅读试卷首页的试题说明

(1) 时间复杂度推测：

- n 在 10 以下：可能是暴力搜索/全排列枚举所有情况
- n 在 20 以下：可以考虑二进制枚举 $O(2^n)$
- n 在 40 以下：可能是搜索+优化，比如折半查找等
- n 在 100~500：$O(n^3)$
- n 在 1000~10000：$O(n^2)$
- n 在 10^5~10^6：$O(n\log n)$
- n 在 10^6~10^7：O(n)
- n ≥ 10^7：$O(\log n)$ 或 O(1)，如果不超过 10^{16}，也有可能是 $O(\sqrt{n})$等

(2) 检查空间限制及“0”个数的检查

很多同学会在定义大数组时多 0、少 0，都会导致空间问题！一定要数清楚个数！

先找到所有数组，依次计算每个的空间大小：

内存大小 = **数组元素个数*类型字节数/1024/1024**（单位是 MB）

常见类型大小：

- int 类型字节数是 4
- long long 类型字节数是 8
- char 类型字节数是 1
- double 类型字节数是 8

记住 int 型数组在 128MB 内存下最大不要超过 $2.5*10^7$ 是比较保险的(占 100MB 内存)。

(3) 检查数据范围：int 和 long long

int 类型：[-2^{31}~2^{31}-1]，即[-2147483648,2147483647]

① 遇到**求和、求乘积、求幂**等情况，一定要提前估算结果的数据范围！

② 变量是 long long 的，**函数返回值类型、循环变量**等相关的都检查一下是否也要换成 long long

(4) 数据未清空检查及变量初始化

数组、变量，能放到**全局**尽可能放到全局（main 函数和自定义函数的外部），放到**局部的变量**必须**初始化**！多测一定要清空！

(5) 变量名

不要在全局使用 x1,y1,x2,y2 之类过于简单的变量名，在 windows 下编译可以通过，但在 linux 下却是会编译失败的！

(6) 头文件检查

以下均是在 devC++中能编译通过，在 linux 下编译错误的万能头文件：

```
#include<bits/stdC++.h>
```
✗

```
#include<bits\stdc++.h>
```
✗

(7) 检查注释代码

再次检查是否有输出调试的语句，调试语句未注释也会导致答案错误！同时再次检查是否忘记取消 freopen 注释！

调试相关

1. 大样例的检查与比较（windows 下的 fc 命令）

假设有一组大样例：apple3.in 和 apple3.ans，输入输出都很大，无法肉眼比较。我们可以按下面步骤操作：

(1) 在自己的代码里使用 freopen 读入 apple3.in，输出到 apple.out，并运行：

```cpp
#include<bits/stdc++.h>
using namespace std;

int main(){
    //apple3.in是输入样例
    freopen("apple3.in","r",stdin);
    //apple.out是自己的输出
    freopen("apple.out","w",stdout);
    //以下是其他代码
}
```

(2) 比对自己的输出 apple.out 和标准输出 apple3.ans 的差异：

① 在这两个文件的同目录下，再新建一个 run.cpp，里面写：

```cpp
#include<bits/stdc++.h>
using namespace std;

int main(){
    system("fc apple3.ans apple.out\n");
}
```

② 运行这份 run.cpp 代码，如果有不同之处，就会提示：

```
正在比较文件 apple3.ans 和 APPLE.OUT
***** apple3.ans
1 2 3
***** APPLE.OUT
1 2 4
*****
```

2. DevC++中的调试工具

如果没学过/不熟悉，正式考试的时候慎用。

debug 开始前，检查是否调整好 devc++：选择 64-bit Debug

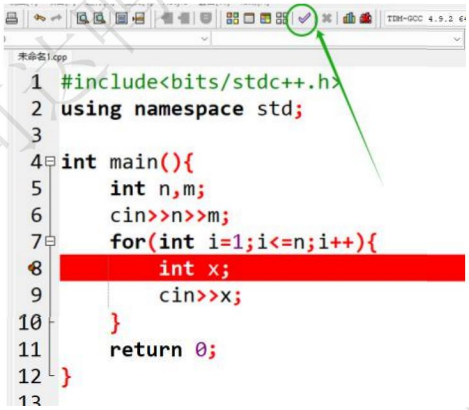


(1) 点击行号、添加断点：

```cpp
#include<bits/stdc++.h>
using namespace std;

int main(){
    int n,m;
    cin>>n>>m;
    for(int i=1;i<=n;i++){
        int x;
        cin>>x;
    }
    return 0;
}
```

(2) 点击调试按钮



(3) 使用工具进行调试：



3. 其他调试方法

(1) 输出中间变量：

代码中插入 cout 语句，通过输出变量的值来观察每一次的执行结果

(2) 死循环/代码崩溃定位：

每隔一段代码、输出一行语句来进行调试，看看到底死循环在哪了。比如下面代码，每隔一段，输出了一行----------，我们就可以锁定是哪段代码出问题。

```cpp
#include<bits/stdc++.h>
using namespace std;

int main(){
    cout<<"--------"<<1<<"--------"<<endl;
    cin>>n>>m;
    for(int i=1;i<=n;i++) cin>>a[i];
    cout<<"--------"<<2<<"--------"<<endl;
    for(int i=1;i<=n;i++) cout<<a[i];
    cout<<"--------"<<3<<"--------"<<endl;
}
```

比如上面的代码，如果输出了--------2--------，代码崩溃/死循环了，那么我们就可以知道是第9行出现了问题。

(3) 手动构造样例：

① 边界情况：手动构造处于边界值时的小样例并手推结果验证

② 特殊情况：根据题目中数据范围给出的特殊性质构造

③ 构造大样例：写一个 gen.cpp 造数据，使用随机函数进行构造与数据范围相同的一些输入值，使用 freopen 写入一个.in 文件中，再用自己的标准程序读入这个.in 文件，对比结果

使用 rand()生成随机数的方法：

```cpp
#include<bits/stdc++.h>
using namespace std;

int main(){
    srand(time(0));
    int x=rand();
    cout<<x;
}
```

如果要生成某个值范围的数，可以用取余的方法：

```cpp
#include<bits/stdc++.h>
using namespace std;

int main(){
    srand(time(0));
    //生成0~9的随机数
    int x=rand()%10;
    //生成10~19的随机数
    x=rand()%10+10;
}
```

注意：rand()前要取种子 srand(time(0))，windows 下的随机数范围是$[0,2^{15}-1]$，要生成 int 范围的可以用(rand() << 15 | rand())，生成 long long 的同理。

如果是 linux 下，rand()的范围是$[0,2^{31}-1]$

常见报错案例

1. 常见编译错误（介绍一些报错信息的单词）

(1) [Error] oistream: No such file or directory

可能的错误原因：#include<oistream>

头文件写错

(2) [Error] 'a' was not declared in this scope

可能的错误原因：a 这个变量并未声明，但是使用了。

(3) [Error] expression cannot be used as a function

可能的错误原因：int c=3(a+b);

忘了写运算符导致的

(4) [Error] redeclaration of 'int a'

可能的错误原因：对变量 a 重复定义了

(5) [Error] 'else' without a previous 'if'

可能的错误原因：else 前面找不到对应的 if 与之匹配。写嵌套的 if-else 语句时，尽量都写上大括号。

一个常见错误举例：

想写的逻辑：

```
if(a>b){
    if(b>c){
        cout<<"a is the biggest"<<endl;
    }
}
else cout<<"a is not the biggest"<<endl;
```

忽略括号的错误写法：

```
if(a>b)
    if(b>c)
        cout<<"a is the biggest"<<endl;
else cout<<"a is not the biggest"<<endl;
```

可以发现，else 实际上与 if(b>c)配对了，这样不会产生编译错误，但是逻辑会有错。

(6) [Error] stray '\243' in program

可能的错误原因：代码中存在中文符号，比如中文的分号等

(7) [Error] no matching function for call to 'max(int&, long long int&)'

可能的错误原因：两个不同数据类型的变量，使用 max 函数进行比较了。min 函数同理，也不能这样

(8) `[Error] invalid conversion from 'const char*' to 'char'[-fpermissive]`

可能的错误原因：字符串赋值给字符了。注意，双引号代表字符串，也就是 const char* 类型，单引号是字符类型，也就是 char 类型，两者是不同的。

(9) 跳到另一个代码报错，并伴随一大堆报错信息



可能的错误原因：一般和结构体有关，比如：

```
#include<bits/stdc++.h>
using namespace std;
struct node{
    int a,b,c;
}a[10005];
int main(){
    sort(a+1,a+11);
    return 0;
}
```

代码中没有未结构体编写 cmp 函数或重载小于号<

2. 常见爆零例子

(1) 文件读写相关错误

① 没写文件读写、文件名字写错了，或文件名后多了数字、加了空格

② 把输入输出写在了 freopen 前。

(2) 读入输出相关错误

① cerr 没删虽然没 WA 但是导致 TLE。

② 没关闭同步流，使用 cin,cout 导致 TLE。

③ 关闭同步流后， cin,cout 与 scanf,printf 不能混用。

④ 关闭同步流后，没有 endl 清空缓冲区然后 fclose 了，导致没有输出。

⑤ 关闭同步流后，没有 #define endl '\n' 导致反复清空缓冲区最后 TLE。

⑥ scanf 普通变量没有加 &，读字符数组的时候加入了 &。

⑦ printf 输出 long long 的时候用了%d 而不是%lld，输出 double 的时候用 %f 而不是 %lf。

(3) 命名相关错误

① x1,y1,y0,y1,yn,j0,j1,jn 都不能当变量名使用。

② 使用了 next,time,case,array 等完整单词作为变量名。

③ 相同的变量名在全局和局部都开了一次。比如在全局定义了 n，又在 main 函数里定义了 n。这种情况不会产生编译错误。

(4) 写顺手了

① 题目先读入 m,n，但是写顺手了 cin>>n>>m。

② 内外层循环都用了同一个变量 i

③ 写顺手了 for(int i=n;i>=1;++i)

④ 写了 if (...); 后顺手加了个分号。

⑤ 写顺手了 for(int i=1;i<=n;++i)for(int j=1;j<=n;++i)

(5) #define int long long 相关：建议不要这样写！

① 使用了 #define int long long 导致 TLE 或者 MLE 或者 RE。

② 使用了 #define int long long 后没有改为 signed main()。

③ 使用了 #define int long long 后 scanf 用的还是 %d。

(6) 代码习惯不好相关

① =和==不分。

② 一个局部变量没初始化就使用。

③ 运算符**优先级**记不清时没打括号

④ 使用 STL 容器没有先**判空**就 pop 或者 top。

⑤ 禁止把大数组开在局部里。

⑥ 没有想清楚下标从 0 开始还是从 1 开始。

⑦ 函数里不写 return。如果**有返回值函数**没有写 return 会导致测评 RE，但本地测试时不会发现错误。

⑧ 递归没写**递归边界**导致 RE。

⑨ double(int/int) 算出来还是个整数。

⑩ 及时取模，不要等所有的运算都结束之后再取模。

⑪ 某年 NOI 的真实事件：有人比赛结束了，但是代码编辑器没有关，然后不小心压到了键盘，并且自动保存了。

⑫ 要记得随手 Ctrl+S 保存

(7) 坑点提示

① 禁止使用 gets。

② 判断一个数的奇偶不要用 if(x%2==1)，要用 if(x%2)，因为 x 有可能是负数，模完是 -1。这种特殊情况要考虑到。

③ 用位运算判断偶数不要写 if(x&1==0)，因为会先算 1==0，要打括号：if((x&1)==0)。

④ 2 的 60 次方是 1ll << 60，不是 1<<60。

⑤ getchar 有可能会读入换行符。

⑥ sqrt 在 long long 范围内有可能产生精度误差。

⑦ 注意题目的 YES 还是 Yes 还是 yes 还是 YE5。

(8) STL 相关坑点

① 谨慎开很多个 deque、queue、stack，会导致 MLE+TLE

② 不要用 if(m[x]==0)这种写法来判断 x 在不在 map 里，用 if(m.count(x))来判断

③ 没有使用 set.lower_bound 而是 lower_bound(set.begin(),set.end()) 导致 TLE。

④ 使用 multiset 然后 erase 了一个值导致全被删了。删除一个 x，需要先使用 find 或 lower_bound 找到对应元素的迭代器，再把这个迭代器传入到 erase 函数中进行删除。

⑤ priority_queue 牢记是**大根堆**。如果实在是忘了，可以找个代码做实验，把 1 2 3 4 5 插入，看谁在堆顶）。

⑥ size() 是一个**无符号类型**，不要拿它做减法。

⑦ 在进行访问、删除操作前，没有判断容器是否为**空**。

(9) 线段树相关

① 线段树要记得开 4 倍空间。

② lazy 标记没有清空。

③ 线段树节点信息修改后没有 pushup。

④ 带 lazy 标记忘了 pushdown。

⑤ 查询修改之类的，注意判一下 l>r 的情况。

⑥ 计算 mid 的时候注意 l + r 是否为负数，是负数的话向下取整用(l+r)>> 1，否则(l+r)/2

会向零取整

3. 题目中的 corner

(1) 除 0 模 0。遇到相关题目一定要思考一下有没有这种可能性。

(2) 没有特判 n=0，n=1 之类的 corner 数据。

(3) 有关质数的题没考虑到 2 是唯一的一个偶数质数。

(4) 有关图的题，题目到底有没有说清楚有没有自环和重边。

进阶知识

1. 进阶技巧：对拍

如果没学过/不熟悉对拍，正式考试的时候慎用，用之前**首先要保证前两题能做出来，因为对拍会花费大量的时间**。

(1) 找出使代码错误的数据

首先我们设定几个在同一目录下的文件：

- std.cpp：把要验证的正解代码放到这个文件中，并编译出 exe 文件。
- brute.cpp：保证正确的暴力解法代码，放到这个文件中，并编译出 exe 文件。
- gen.cpp：按照题目要求生成一组合法的样例，并编译出 exe 文件
- run.cpp：利用上面三个代码文件，开始验证。

run.cpp 举例：

```
int main(){
    int t = 100000;//对拍执行100000次
    while (t--){//执行对拍
        system("gen.exe>data.in");//运行gen.exe生成一组数据，输出到data.in文件中
        system("brute.exe<data.in>brute.out");//运行brute.exe，从data.in输入，结果输出到brute.out
        system("std.exe<data.in>std.out");//运行std.exe，从data.in输入，结果输出到std.out
        if (system("fc brute.out std.out"))//比较暴力结果和正解结果是否相同
            break;//不相同，跳出
    }
    if (t == 0)
        cout << "no error" << endl;
    else
        cout << "error" << endl;
    return 0;
}
```

如果暴力解法和正解解法产生了差异，那么 data.in 里面就是一个反例，brute.out 里面就是正确答案，std.out 里面就是错误答案。针对这个反例再去分析问题，调整代码。

对拍的过程可能比较漫长，可能需要较长时间才能找到一个错误例子，可以把程序运行之后挂到一旁，去检查、验证其他题目。

(2) 检查代码是否超时

对拍的结果是正确后，要生成大样例计算平均耗时。需要把 gen 改成生成大样例，单独跑 std。跑 1000 组大样例，计算这 1000 组的平均耗时。如果平均耗时超过题目要求，需要进一步优化代码。

```
int cost = 0;
for (int i = 1; i <= 1000; i++) {
    system("./gen >in"); // 只生成大样例
    auto st = clock();  // 记录std开始时间
    system("./std < in > std.out");
    auto ed = clock(); // 记录std结束时间
    cost += ed - st;
}
cout << cost / 1000;
```

优化的代码命名为 std2。用 std 和 std2 去对拍大样例，记录 std2 的平均耗时，看 std2 的平均耗时是否满足题目要求。如果满足要求，就将 std2 重命名为提交的文件名。

```
int cost = 0;
for (int i = 1; i <= 1000; i++) {
    system("./gen >in"); // 只生成大样例
    system("./std <in >std.out");
    auto st = clock();  // 记录std2的开始时间
    system("./std2 < in > std2.out");
    auto ed = clock(); // 记录std2的结束时间
    if (system("diff std.out std2.out")) { // 还是要检查一下，std2是否写错
        cout << "WA\n";
        return 0;
    }
    cost += ed - st;
}
cout << cost / 1000;
```

即使只会部分分，想不到正解 100 分的做法，对部分分代码也可以对拍，能够确保部分分是正确的。

2. 常见的判题机的判题逻辑

(1) 传统型

一个.in 文件和一个.out 文件组成一个数据点，用户的代码提交之后，判题机编译并运行，将一个数据点的.in 文件输入，得到输出结果，和这个数据点的.out 文件比较，如果相同，判定为通过这一个数据点。一道题目往往有好多个数据点组成。比如一个 100 分的题目，有 20 个数据点，如果是均分分数的话，通过一个数据点就可以得到 5 分。

(2) special judge

如果一个题目有多解，传统型就难以应对了，因为不可能穷尽所有的答案。

举个例子，比如题目要求输出一个大于 3 的数字，那么 spj 里面，可以写用户输出结果如果大于 3，那么就判定这个数据点通过。

special judge 就是在传统型的基础上，在最后阶段，不是直接比较用户输出结果和.out 文件是否相同，而是直接把信息都传给一个 spj 可执行文件，由 spj 来决定是否通过。

考试策略与心态调整

1. 良好的行为习惯
   (1) 进入考场后先别点开下发文件，先**复制备份一份**。防止后续把题目数据破坏。
   (2) 不是特别确定的算法或者不是特别熟悉的代码模板不要写。
   (3) 读题包括读数据范围分析空间复杂度和时间复杂度，很多情况下会漏掉空间大小分析。
   (4) 注意使用数组不要下标越界，数组开太小或开太大，把 0 的个数数清楚。元素个数酌情多开 3 或 5 或 10 个，防止越界。
   (5) 调试的代码记得注释掉
   (6) 不要炫技，越复杂的程序容易出错的地方越多，朴素 > 高调。
2. 时间分配
   (1) 看题策略：前 30 分钟通读题目，优先写简单题。不要纠死在一道题。
   (2) 经常要看时间，记清楚考试结束时间。
   (3) 比赛还剩 30 分钟的时候，如果有还没有结束的代码，快速想办法完成。
   (4) 比赛结束前 10 分钟，不要继续写代码了。应该检查一下你的文件读写，再编译一遍，测试一下确保能过大样例，以及之前写的部分分解法能够拿到分。
3. 做题策略与骗分
   (1) 暴力打满就是省一。
   (2) 一道题怎么都没思路，先确定一下是不是题读错了，或者看漏条件了。
   (3) 如果一道题目，30 分钟还在纠结思路或者没有思路，果断放弃满分的做法，开始写部分分。
   (4) 如果代码怎么都调不出来，那就删了重写一遍。
4. 写代码的注意事项
   (1) 一定想好算法、思路清晰了，再去编程。不要知道个大概就开始写，最好是打好草稿，避免写的过程中忘记特殊情况或本该需要注意的细节。
   (2) 写代码注释，写清变量意义、程序步骤。对于一些比较复杂的题目，这一点是必需的。一定要仔细，不要着急。
   (3) 提前出好测试数据，极大值、极小值、思路可能出错的、大数据(代码生成 写进文件)。
   (4) 在不超限制的前提下，能不优化就不优化，以减少代码量和出错概率为第一原则。
   (5) 永远别在考场上写从未接触过的算法/数据结构。
   (6) 针对某些特殊的量，可以适当打表节省时间，但是注意打表不要太长，代码文件的大小不要超过

50KB。

5. 心态调整

赛前

(1) 不用紧张，不用害怕。

(2) 你的训练是不会背叛你的。

(3) 你的对手大多都是一群乌合之众，他们大多数连通过编译都是个问题。

(4) 不用把比赛看的太重，你未来还会参加无数的比赛。

(5) 考前一定要保证充足的睡眠，保持平稳的心态。不要自我加压，正常发挥就好。

赛中

(1) 很多题不可能马上就想出解法，要敢于钻研，想到一定时候便想出来了。不要怕，其实很多时候你是有能力做的。

(2) 题难，大家都难，不要怕。

(3) 不要上来发现四个题都不会心态就炸掉，四个题能被你一眼看出来做法才是不合理的。

(4) 如果发现题很简单，或者你很擅长，不要过度激动，因为乐极生悲。要稳重严肃认真的对待每一道题，狮子搏兔尚尽全力。

(5) 不要一直钻研一道题，可能隔一段时间再看就会有思路。

(6) 不要因为老师家长给予了很大的期望而害怕让他们失望，考试过程中只专注做题，不要想其他。

(7) 永远不要过度相信自己，墨菲定律永远是存在的。



撷码可达鸭编程

XIE MA CODUCK

联系方式：

15121058662（微信同号）

公众号：

信息学算法编程

扫码添加微信

扫码关注公众号